

Deep Learning Fundamentals

Membangun neural network dengan TensorFlow/Keras: dari NN dasar (Sequential, Dense), fungsi aktivasi & optimizer, CNN & transfer learning untuk gambar, RNN/LSTM/GRU untuk data berurutan, hingga akselerasi GPU NVIDIA (mixed precision, TensorRT), plus intro generative AI (autoencoder & GAN).

Alur: Siapkan data: load dataset, normalisasi pixel (bagi 255 -> 0.0-1.0), pisah train/val/test → Bangun model: tf.keras.Sequential tumpuk layer (Flatten/Dense, Conv2D, LSTM, dll) → compile: tentukan loss, optimizer (Adam), dan metrics=[accuracy] → fit: latih beberapa epoch dengan batch_size + validation_data (cek overfitting) → evaluate: uji di test set yang belum pernah dilihat (akurasi, confusion matrix) → predict: Softmax multi-kelas -> argmax (kelas prob tertinggi); Sigmoid biner -> threshold 0.5 → Akselerasi (opsional): jalankan di GPU, mixed_float16, optimasi inference dengan TensorRT

Neural Network: blok dasar

- Neuron = unit pemrosesan terkecil; Layer = kumpulan neuron
- Weights (bobot) = kekuatan koneksi yang DIPELAJARI saat training
- Alur: input -> hidden layers -> output (prediksi)
- NN belajar pola dengan menyetel ribuan weights, bukan aturan manual

► Fondasi semua arsitektur deep learning

Model Sequential (Dense)

- Sequential = tumpukan layer berurutan input -> output
- Flatten ratakan 28x28 jadi 784 angka
- Dense = fully-connected; hidden pakai activation='relu'
- Output Dense(10, softmax) -> probabilitas 10 kelas (Fashion MNIST)

```
model = tf.keras.Sequential([
    layers.Flatten(input_shape=(28,28)),
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')])
```

► Klasifikasi/regresi data tabular atau gambar yang sudah diratakan

Training: compile + fit

- compile butuh: loss, optimizer, metrics
- Loss (crossentropy) = ukur seberapa salah; optimizer perbaiki weights
- Epoch = 1 putaran lihat SEMUA data, diproses per batch
- Target: loss turun, kurva train & val rapat (hindari overfitting)

```
model.compile(loss='sparse_categorical_crossentropy',
               optimizer='adam', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=20,
          batch_size=32, validation_data=(X_valid,
                                           y_valid))
```

► Setelah model dibangun, sebelum evaluasi

Kenapa perlu Activation?

- Tanpa activation NN cuma jumlah & kali (linear)
- Banyak layer linear ditumpuk = tetap 1 fungsi linear
- Activation tambah non-linearity -> kenali pola berkelok
- Tanpa activation, 100 layer = 1 layer linear belaka

► Memahami mengapa setiap hidden layer butuh fungsi aktivasi

Fungsi Aktivasi utama

- ReLU max(0,x): cepat, standar industri di hidden layer
- Sigmoid (0-1): output biner Ya/Tidak
- Tanh (-1-1): zero-centered, stabil
- Softmax: output multi-kelas; semua probabilitas jumlahnya 100%
- Risiko: vanishing gradient & dead neuron (LeakyReLU bantu)

► Pilih per layer: ReLU hidden, Sigmoid biner, Softmax multi-kelas, Linear regresi

Optimizer

- Optimizer = cara update weights via gradient (turun gunung berkabut)
- SGD+Momentum: langkah kecil, inersia stabil
- RMSprop: sesuaikan langkah; cocok RNN/LSTM
- Adam: adaptif arah + kecepatan, paling populer (default)

```
optimizer='adam'
```

► Mulai selalu dengan Adam; pindah SGD+Momentum bila tak stabil

CNN untuk gambar

- Convolution: filter/kernel 3x3 pindai gambar cari pola lokal (tepi, tekstur)
- Pooling (MaxPooling2D): susutkan ukuran, simpan fitur penting
- Weights di-share -> parameter jauh lebih sedikit
- Belajar bertingkat: tepi -> bentuk -> bagian objek
- Praktik: CIFAR-10 (60K gambar warna 32x32, 10 kelas)

```
Conv2D(32, (3,3), activation='relu',
       padding='same', input_shape=(32,32,3))
MaxPooling2D() # 32->16
```

► Klasifikasi/deteksi citra

Transfer Learning (ResNet50)

- Pakai model pretrained ImageNet (14 juta gambar) yang sudah pintar
- Freeze layer bawah: base.trainable = False
- Ganti head: pooling -> Dense -> Dense(10)
- Unggul saat data berlabel terbatas; latih cepat (layer di-freeze)

```
base = ResNet50(weights='imagenet',
                 include_top=False, input_shape=(32,32,3))
base.trainable = False # freeze
```

► Data terbatas / gambar resolusi tinggi -> jalan pintas standar industri

RNN / LSTM / GRU (data berurutan)

- Sequential data: urutan PENTING (teks, saham, cuaca)
- RNN ingat langkah sebelumnya via hidden state (h)
- Masalah RNN: vanishing gradient -> lupa konteks panjang
- LSTM punya cell state + 3 gates (forget/input/output) = memori panjang
- GRU: 2 gates (update/reset), seimbang cepat & akurat

```
Embedding(vocab_size, 64, input_length=200) #
kata->vektor 64-dim; panjang sequence 200
(padding)
LSTM(64)
Dense(1, activation='sigmoid') # sentimen IMDB
```

► Teks, deret waktu, sinyal; urutan punya makna

Akselerasi GPU NVIDIA

- CPU serial (sedikit core kuat) vs GPU paralel (ribuan core)
- Inti NN = matrix multiplication -> sangat paralel, GPU ~10-100x lebih cepat (ilustratif, tergantung ukuran/operasi)
- Mixed precision: float16 untuk forward/backward (Tensor Cores), float32 untuk master weights -> hemat memori ~50%
- TensorRT optimasi model terlatih: layer fusion + precision calibration -> inference 2-5x lebih cepat (TF-TRT, FP16)

```
from tensorflow.keras import mixed_precision
mixed_precision.set_global_policy('mixed_float16')
Dense(10, 'softmax', dtype='float32') # output FP32
```

► Latih/inference model besar dengan cepat dan hemat memori

Generative AI: AE & GAN

- Model generatif menghasilkan DATA BARU, bukan menebak label
- Autoencoder: encoder -> latent space -> decoder; decoder = generator
- GAN: generator (pemalsu) vs discriminator (kurator) dilatih bergantian
- Modern: diffusion (Stable Diffusion) & autoregressive (GPT) -> Module 04

► Fondasi ChatGPT & Stable Diffusion

Quick-Ref: Layer, Optimizer, Model & Dataset

Item	Pakai untuk	Catatan kunci
Dense	Fully-connected layer	Hidden activation='relu'; output sesuai tugas
Conv2D	Ekstrak fitur gambar	Filter 3x3, activation='relu', padding='same'
MaxPooling2D	Susutkan dimensi	32->16->8->4 sambil filter bertambah
Dropout(0.5)	Cegah overfitting	Matikan sebagian neuron acak saat training
SimpleRNN	Urutan pendek (<10 langkah)	0 gates, cepat tapi mudah lupa
LSTM	Urutan panjang, teks, musik	3 gates + cell state; ~4x parameter, lebih lambat
GRU	Seimbang cepat & akurat	2 gates (update/reset)
Adam	Optimizer default	Adaptif; selalu coba dulu
RMSprop	Optimizer untuk RNN/LSTM	Sesuaikan ukuran langkah
Fashion MNIST	Latihan NN (Act 1)	70K gambar 28x28 grayscale, 10 kelas, ~88% acc
CIFAR-10	Latihan CNN (Act 3)	60K gambar warna 32x32, 10 kelas
IMDB	Latihan LSTM sentimen (Act 4)	50K review, Embedding 64-dim, pad 200 kata

Isilah Kunci

Weights (bobot) — Nilai kekuatan koneksi antar neuron yang dipelajari dan dioptimalkan selama training.

Activation function — Fungsi non-linear (ReLU, Sigmoid, Tanh, Softmax) yang membuat jaringan bisa mengenali pola kompleks.

Loss — Ukuran seberapa salah prediksi model; training berusaha menurunkannya (mis. crossentropy, mse).

Optimizer — Algoritma update weights memakai gradient (Adam, SGD+Momentum, RMSprop).

Epoch / batch — Epoch = 1 putaran melihat semua data; data diproses per batch (mis. batch_size=32).

Overfitting — Model hafal data latih tapi buruk di data baru; terlihat saat kurva train & val melebar.

Hidden state — Memori RNN yang diteruskan antar timestep agar informasi langkah sebelumnya tersimpan.

Mixed precision — Latih dengan float16 (cepat, Tensor Cores) + float32 (master weights) untuk hemat memori ~50%.

Generator — network yang mengubah noise menjadi data baru (gambar/teks)

Discriminator — network penilai pada GAN: asli atau palsu

Latent space — ruang representasi terkompresi tempat data "diringkas"

Diffusion — model generatif yang mencipta dengan denoising bertahap

